Appunti Tecnologie e Applicazioni Web

Giacomo Bordignon

Febbraio 2025

Contenuti

1 World Wide Web **5**
- 1.1 Browsing the web . . . 5
- 1.2 Storia del World Wide Web . . . 6

2 Il protocollo HTTP **8**
- 2.1 Resource Identifier . . . 8
- 2.2 Request-Response model . . . 10
- 2.3 Status Code . . . 10
- 2.4 HTTP nel dettaglio . . . 10
  - 2.4.1 HTTP messages . . . 11

3 Il protocollo HTTPS **13**
- 3.1 Feature di sicurezza . . . 13
- 3.2 Crittografia . . . 13
  - 3.2.1 Firma digitale . . . 13
  - 3.2.2 Certificato digitale . . . 14
- 3.3 Connessione . . . 14
- 3.4 Sessione . . . 14
  - 3.4.1 Tipi di cookies . . . 15
  - 3.4.2 Cross-site scripting (XSS) . . . 15
  - 3.4.3 Cross-site request forgery (CSRF) . . . 15
  - 3.4.4 Cookie-header . . . 16
  - 3.4.5 Cookie, security e privacy . . . 16

4 Autenticazione **17**
- 4.1 Basic Access Authentication . . . 17
- 4.2 JSON Web Token (JWT) . . . 18
- 4.3 JSON . . . 19
  - 4.3.1 JSON Objects . . . 20
  - 4.3.2 JSON arrays . . . 20

5 Web Socket **20**
- 5.1 Web Socket Handshake . . . 21

6 REST APIs **22**
- 6.1 Risorse . . . 23
  - 6.1.1 Resource identifier . . . 23
- 6.2 API Design . . . 23
  - 6.2.1 Semantic versioning . . . 24

7 Docker Containers **25**
- 7.1 Architettura docker . . . 25
- 7.2 Creazione di un'immagine e di un container in docker . . . 26
- 7.3 Single-Host Networking . . . 26
- 7.4 File system dei container . . . 26

7.5 Copy-on-write . . . 27
7.6 Volumi . . . 27
7.7 Creare e gestire immagini . . . 28

8 Javascript **28**
8.1 Dettagli di Javascript . . . 28
8.2 Tipi di dato . . . 29
8.3 Oggetti in Javascript . . . 30
8.3.1 Oggetti nel dettaglio . . . 30
8.3.2 Accesso alle property . . . 31
8.3.3 Global object . . . 31
8.3.4 Creazione di un oggetto . . . 31
8.3.5 Delete property . . . 32
8.3.6 Spread operator . . . 33
8.4 Funzioni di Javascript . . . 33
8.4.1 Definizione di funzione . . . 33
8.4.2 Invocazione di una funzione . . . 34
8.4.3 Keyword this . . . 34
8.4.4 Funzioni arrow . . . 34
8.4.5 Variadic function . . . 35
8.5 Chiusure (closure) in Javascript . . . 35
8.6 Classi in Javascript . . . 36
8.6.1 Costruttori . . . 36
8.6.2 Sottoclassi . . . 36
8.7 Stringhe in Javascript . . . 37
8.8 Statement . . . 37
8.9 Variabili . . . 38

9 Typescript **38**
9.1 Javascript e typescript . . . 39
9.2 Feature di Typescript . . . 39
9.3 Type annotation . . . 39
9.4 Tipi base . . . 39
9.5 Interfacce . . . 40
9.5.1 Implementazione interfacce . . . 40
9.6 Sottotipi strutturali . . . 40
9.7 Type assertion . . . 41

10 NodeJS **41**
10.1 Moduli . . . 42
10.2 npm . . . 42
10.2.1 Operazioni asincrone . . . 43
10.2.2 Codice single-thread . . . 43
10.2.3 Concorrenza condivisa . . . 43
10.2.4 NodeJS APIs . . . 43

10.2.5 Considerazioni su un esempio di progetto (Node.js simple http server) . . . 44

11 MongoDB **46**
11.1 Promises vs Callbacks . . . 47

12 ExpressJS **48**
12.1 Routing . . . 48
12.2 Static pages . . . 48
12.3 Autenticazione con Express.js . . . 49

13 Angular **50**
13.1 MVC . . . 50
13.1.1 Model . . . 50
13.1.2 Controller . . . 51
13.1.3 Template (View) . . . 51
13.2 Decorators . . . 51
13.3 Architettura di Angular . . . 52
13.4 Angular Components . . . 52
13.5 Data Binding . . . 52
13.6 Built-in directives . . . 53
13.7 Comandi per installare Angular . . . 53
13.8 Dependency injection . . . 54
13.9 Observables . . . 55
13.9.1 Differenza tra Observables e Promises . . . 55
13.10 HttpClient . . . 55
13.11 Routing . . . 56
13.11.1 Setup routing . . . 56

1 World Wide Web

Prima di parlare di protocolli è utile chiarire alcuni concetti chiave. Tra questi possiamo trovare:

- **INTERNET:** Sistema composto da molte reti, che usa dei protocolli per connettere i vari dispositivi nel mondo. Ogni nodo è identificato da un indirizzo ed è riferito ad un host. E' possibile comunicare anche tra due dispositivi non direttamente connessi; per fare questo servono dei PROTOCOLLI, definiti da uno STACK DI PROTOCOLLI che mette d'accordo su quale "linguaggio" usare per comunicare.
- **WORLD WIDE WEB:** Sistema di informazioni che servono per **identificare** le **risorse** accessibili in Internet tramite un **URL** associato.

Le risorse sono tipicamente pagine web collegate tra loro e vengono dette **hypertext**.
Per accedere alle risorse c'è bisogno di un software che viene detto **browser** o **web agent**.
Le basi delle risorse presenti nel web sono:

1. URL indica la posizione univoca di una determinata risorsa, ossia dove si trova nella rete
2. Web agents comunicano con i server usando protocolli standard
3. Le risorse hanno uno **specifico tipo** che ne **rappresenta** il **contenuto**. N.B. **Il tipo della risorsa NON E' dato solo dal nome.** Inoltre la singola risorsa può avere più tipi contemporaneamente, così da poter fornire i dati in diversi formati.

1.1 Browsing the web

Di seguito vengono indicate quali sono e come funzionano le varie fasi della ricerca di una determinata risorsa nel web.

1. Tutto inizia da un web browser e da un URL (ad esempio http://www.example.com/home.html)
2. L'URL viene diviso in:

   http : **indica il protocollo utilizzato**, spesso nel caso di pagine web è HTTP/HTTPS
   www.example.com : **indirizzo del host che identifica il dispositivo su cui è presente la risorsa**. Al momento della ricerca il nome del host viene risolto in un indirizzo IP attraverso il protocollo DNS (Domain Name System), perchè per le persone è molto più semplice ricordare il nome piuttosto che l'indirizzo del server in cui è ospitata tale risorsa.
   home.html: **nome della risorsa**

3. Il browser crea una connessione TCP con altri host detti **server** e inizia la comunicazione usando il protocollo specificato nell'URL. In questo caso HTTP protocol.

4. La risorsa, come già accennato, ha un tipo associato detto **mime-type** perchè il browser deve sapere come interpretare tale risorsa al fine di visualizzarla correttamente

5. Infine il browser interpreta la risorsa e la visualizza graficamente

6. Se si tratta di una pagina HTML essa può contenere link riferiti ad altre risorse creando quello che viene definito **web of information.**

1.2 Storia del World Wide Web

Il protocollo HTTP è stato definito nel 1990 e utilizzava un altro particolare protocollo che si chiamava Request-response protocol per condividere hypertext.

HMTL è il linguaggio che **definisce la struttura del documento** (pagina web) ed è un liguaggio **dichiarativo.**

Nel 1993 venne rilasciato il primo web browser MOSAIC.

Nel 1995 nasce Javascript il cui obiettivo di questo linguaggio era di rendere la pagina web dinamica "on the fly" sulla base dell'input fornito dall'utente. Con il tempo diventa un linguaggio di scripting imperativo e integrato nei web browser.

Nel 1996 nasce CSS: che ha come obiettivo quello di specificare come deve essere presentato un elemento a livello grafico. In altre parole separa il contenuto della risorsa dalla sua presentazione.

Quindi riassumendo:

- JAVASCRIPT : modifica il comportamento della risorsa rendendola dinamica
- HTML : definisce la struttura di tale risorsa
- CSS: definisce come deve essere presentata

Tra il 1996-2000 CSS e Javascript diventano tecnologie fondamentali e nacquero cosi le figure dei front-end developer.

FRONT-END DEVELOPER	BACK-END DEVELOPER
User experience e presentazione contenuto	Backstage della pagina (server, db, ecc..)

I framework astraggono molte delle complessità delle app moderne facilitando e velocizzando il loro svilippo, da questo nasce il concetto di "*full-stack developer.*"
VANTAGGI DEI FRAMEWORK:

- Controllo sull'intera applicazione
- Stesse tecnologie possono essere usate per più piattaforme
- Passaggio veloce dall'idea alla realizzazione.

Nel 2005 si iniziò ad utilizzare una nuova tecnologia chiamata **AJAX** (Asynchronous Javascript And XML) che permette alle app di poter comunicare con un server e modificare il loro contenuto in maniera **ASINCRONA**. In altre parole non è più necessario fare il reload della pagina e far ripartire l'interprete, ma le modifiche sono immediatamente visibili.
Queste app vengono dette **Single Page Application** (SPA).

Nel 2008 Google pubblica il suo web browser, Chrome che fornisce:

- compliance con gli standard
- sicurezza tramite i servizi Google integrati,
- compila al momento eseguendo il codice nel linguaggio macchina dell'architettura su cui è installato il browser.

Ad oggi esistono tre versioni di WEB:
WEB 1.0

- Pagine statiche
- Pochi publishers e molti users che ne visualizzano i contenuti

WEB 2.0

- Pagine dinamiche
- Contenuti generati da users per users
- Social Media

WEB 3.0

- Rendere il web accessibile anche alle macchine
- Non è ancora uno standard ed è in fase di studio (reso più difficile dall'uso delle AI generative)

2 Il protocollo HTTP

HTTP è il protocollo utilizzato per la comunicazione sul web.
Non è l'unico protocollo presente, **ma** ancora oggi è molto utilizzato e **ha un ruolo fondamentale**, perchè ha consentito un rapido svluppo in ambito web.

N.B. UN PROTOCOLLO DEFINISCE UNA SEMANTICA E UN SET DI REGOLE PER SCAMBIARE DATI TRA DUE HOST.

HTTP è un protocollo definito request-response e si basa su una comunicazione che utilizza un modello specifico chiamato **CLIENT-SERVER MODEL.** Questo modello prevede che il client faccia una richiesta al server e il server risponde con la risorsa richiesta, se essa è presente.

N.B. LA COMUNICAZIONE INIZIA SEMPRE DAL CLIENT.

Un tipico client è il web browser, consente all'utente di inserire l'URL e stabilire una connessione con il server per ottenere delle risorse.
L'**HTTP server** risponde alla richiesta del client e ogni comunicazione viene **isolata** dalle altre.

Le risorse presenti nel server possono essere:

- **file statici** : presenti nel file-system del server
- **risorse dinamiche** : risorse generate "on the fly" specifiche per quel client.

Ogni risorsa ha un tipo associato detto **MIME TYPE** e non ha niente a che fare con il nome, infatti esso da un'idea del tipo di una risorsa ma non lo definisce.
Il MIME TYPE è una stringa che indica il tipo associato a tale risorsa.
Questa stringa è composta da : ⟨**type**⟩/⟨**subtype**⟩
che rispettivamente indicano il **tipo** e il **sottotipo** di quella risorsa. (Es: text/html oppure text/plain)
Il tipo della risorsa è **utile al client** per **capire come interpretare tali risorse** una volta scaricate dal web server.

2.1 Resource Identifier

Ogni risorsa ha un nome detto **URI** (Uniform Resource Identifier) che serve per identificarla univocamente.
Esistono 2 tipi di URI:

- **URN** : Identifica la risorsa specificando un nome univoco;
- **URL**: definisce la posizione in cui si trova la risorsa (spesso viene scritto come fosse un percorso, simile a quello del file-system)

UNIFORM RESOURCE LOCATOR (URL)
Nome standard per definire e identificare le risorse nel web.
Struttura URL:
$< scheme >: // < user >:< password > @ < host >:< port > / < path >;< params >? < query > \# \ < frag >$
Dove:

- ⟨scheme⟩ indica il protocollo utilizzato per scaricare la risorsa (HTTP/HTTPS)(Obbligatorio)
- ⟨user⟩ ⟨password⟩ usato per autenticarsi nel server. Se non specificato di default è "anonymous". (Opzionale)
- ⟨host⟩ Può essere un hostname risolto dal protocollo DNS in indirizzo IP. L'host deve essere sempre specificato.(Obbligatorio)
- ⟨port⟩ Porta del server che è in ascolto per una connessione. Se omessa la default port è quella usata dal protocollo. HTTP : 80, HTTPS : 443 (Opzionale)
- ⟨path⟩ Il nome della risorsa ospitata nel server. In HTTP il path può essere diviso in segmenti e la forma è quella del path del file-system.(Obbligatorio)
- ⟨params⟩ Posso dare info aggiuntive del tipo key-value al path specificato (Opzionale)
- ⟨query⟩ Simile a ⟨params⟩ con nome e valore separati da & .Da dati aggiuntivi alla risorsa generata.(Opzionale)
- ⟨frag⟩ Identifica un frammento all'interno della risorsa.(Opzionale)

Esistono inoltre 3 tipologie di URL:

- **URL assoluti**: sono totalmente specificati in tutti i campi
- **URL relativi**: specificano l'ultima parte dell'URL obbligatoria, ossia il path della risorsa. Questa tipologia è molto usata nei file HTML.
- **BASE URL** : posso definirlo in due modi: Specifico il Base URL utilizzando il tag ⟨base⟩ nell'header del documento, oppure utilizzo l'URL assoluto che al suo interno contiene anche quello relativo.

In origine l'URL permetteva l'inserimento di solo caratteri ASCII, però ad oggi c'è un modo per inserire anche caratteri non presenti nell'ASCII come le lettere accentate. Per fare ciò è possibile utilizzare dei caratteri detti **escape**. Un esempio di carattere escape è il % . La codifica di default utlizzata dai browser è la UTF-8.
Però questa possibilità ha introdotto alcuni problemi di sicurezza come IDM homograph attack, che consiste nel utilizzare caratteri simili a quelli ASCII nei link per indirizzare a webpage malevoli (phishing).

2.2 Request-Response model

Come già accennato prima il protocollo HTTP usa un modello **request-response**, che si tratta di una **sequenza** di **transazioni** che comprendono un **comando** detto **request**, seguita da una **response** formattata in un **messaggio HTTP**. Il messaggio HTTP può essere a sua volta una request o una response e il suo contenuto è sottoforma di testo "leggibile".

Cosa posso chiedere ad un server?
Esistono 5 HTTP methods che si usano per le richieste ad un server:

- **GET** : ottenere delle risorse
- **POST** : caricare una risorsa sul server (spesso usato per modificare una risorsa già presente)
- **DELETE** : cancellare una risorsa
- **PUT** : caricare una risorsa (spesso risorse intere)
- **HEAD** : simile a GET, ma ottengo solo i metadati

2.3 Status Code

Ogni response deve contenere al suo interno uno status code, ossia un codice di 3 cifre che notificano lo stato della richiesta. Alcuni dei più importanti sono:

- 200 (success)
- 302 (reindirizzamento necessario)
- 404 (la risorsa non esiste)

2.4 HTTP nel dettaglio

Entrando nel dettaglio HTTP è un protocollo che fa parte del layer Applicazione nel modello ISO/OSI.
Esso utilizza il modello TCP/IP per lo scambio dei dati tra client e server.
L'hostname e la porta utlizzata dalla connessione vengono definiti nell'URL, inoltre TCP/IP garantisce:

1. protocollo connection-oriented, una volta stabilita la connessione
2. Reliability (i dati vengono ricevuti correttamente, oppure viene generato un messaggio d'errore)
3. I dati arrivano nello stesso ordine rispetto a come sono stati generati
4. Se uno dei due dispositivi(peer) trasmette ad una velocità maggiore rispetto all'altro, TCP offre una finestra per il trasferimento dei dati (finestra di congestione), così da "aggiustare" la velocità del peer più lento.

La connessione TCP si può definire in 4 parti:
$< sourceIP >< sourceport >< destinationIP >< destinationport >$
Sono ammesse connessioni multiple da e per lo stesso IP oppure per le stesse porte. Ogni connessione ha una quadrupla come quella definita che è diversa in alcune parti rispetto alle altre.
Nel caso di server con richieste da più host, essi possono condividere lo stesso IP di destinazione e la stessa porta di destinazione.
La source port, invece, viene scelta casualmente dal sistema operativo ed è un numero sopra il 1024, perchè non richiede privilegi di amministratore.

2.4.1 HTTP messages

Quando la connessione viene stabilita, il protocollo invia dei messaggi HTTP ciascuno composto da queste 3 parti:

start line	Stringa con il messaggio HTTP
header	Messaggi che descrivono le info della START LINE. Terminano con una riga vuota
body	sezione dove server e client mettono i dati da inviare. In caso di errore sarà vuota

REQUEST MESSAGE:

start line	$< method >< URLrisorsa >< version >$
header	lista di header del tipo $< nomeHeader >< valueHeader >$. Termina con una riga vuota
body	sequenza di byte, oppure text data

Possibili metodi della start line:

- GET : chiede di scaricare la risorsa
- HEAD : chiede al server solo l'header della risorsa
- PUT : chiede al server di salvare la risorsa
- DELETE : chiede al server di cancellare la risorsa, necessaria l'autenticazione

RESPONSE MESSAGE:

start line	$< version >< status >< reasonphrase >$
header	lista di header del tipo $< nomeHeader >< valueHeader >$. Termina con una riga vuota
body	contenuto della risposta

Possibili headers

- GENERAL PURPOSE
- RESPONSE HEADER
- REQUEST HEADER

- ENTITY HEADER
- PROTOCOL EXTENSIONS

GENERAL PURPOSE

Date	data e ora del messaggio
Upgrade	usato per cambiare la versione del protocollo
Cache control	usato dal client per salvare informazioni localmente
Trailer	In caso di upload e download di grandi dimensioni, i dati vengono segmentati

REQUEST HEADER

Client IP	IP address del client (web browser)
Host	Host IP address (indica al server quale IP utilizzare)
Accept	Informa il server quali MIME TYPE supporta
Accept-Encoding	Specifica gli encodings accettati
If-Modified-Since	Chiede al server di inviare solo le risorse modificate dopo una certa data
Authorization	Contiene le credenziali per accedere a risorse riservate
Cookie	Usato dal client per inviare stringhe generiche al server

RESPONSE HEADER

www-Authenticate	Usate per chiedere di fornire authentication data
Set-cookie	Salvare una stringa generica
Server	Nome e versione del server
Allow	Lista di metodi che si può usare sulla risorsa
Content-Encoding	Rescource encoding
Content-Length	Numero di byte del body
Content-Type	MIME TYPE della risorsa
Last Modified	data e ora dell'ultima modifica

Parallel connection: Il client apre molte connessioni TCP in contemporanea e ognuna di queste chiede qualcosa al server. L'unico limite è la banda del client che può limitare l'apertura di molte connessioni.

Oggi molti client mantengono la stessa connessione aperta ed eseguono più richieste una dopo l'altra, questo porta ad un grande consumo di risorse lato server, ecco perchè c'è un **timeout** in cui se non arrivano richieste per un certo periodo di tempo la connessione viene chiusa (HTTP v1.1).

3 Il protocollo HTTPS

HTTPS significa **HTTP Security** in cui **non viene modificato l'originale HTTP** e lavora in un livello intermedio dello stack dei protocolli chiamato **TLS/SSL**.

3.1 Feature di sicurezza

Alcune feature che riguardano la sicurezza sono:

- Data encryption : I dati sono leggibili obbligatoriamente solo da client e server
- Integrità di trasmissione : I dati rimangono integri attraverso la connessione (JSON Web Token)

N.B: Encryption != Integrità

- Server authentication : Il client può verificare l'identità del server per garantirne la sua autorità
- Client authentication : Il server può verificare l'identità del client per garantirne la sua autenticità (certificati digitali).

3.2 Crittografia

Può essere:

- Crittografia a chiave simmetrica : stessa chiave per criptare e decriptare
- Crittografia a chiave asimmetrica : una chiave privata e una chiave pubblica
- Firma digitale
- Certificato digitale

N.B: encryption = messaggio + chiave pubblica ricevente
decryption = messaggio + chiave privata ricevente
Questa tecnica risolve il problema dello scambio delle chiavi, mentre la precedente risolve il problema della data encryption.

3.2.1 Firma digitale

Lavora in maniera **opposta** rispetto alla crittografia a chiave asimmetrica:

1. Il server firma un documento con la sua chiave privata;
2. Per verificare l'identità del server, il ricevente decripta la firma del server con la chiave privata del server stesso;
3. La firma digitale è usata per l'integrità del messaggio.

3.2.2 Certificato digitale

I documenti elettronici sono usati per dimostrare il possesso di una chiave pubblica.
Essa contiene:

- Nome del soggetto (persona, server, organizzazione, ecc..)
- Data di scandenza
- Informazioni sull'emittente (signing authority)
- Chiave pubblica del soggetto

3.3 Connessione

Di seguito si riporta uno schema di come funziona ad alto livello una connessione con l'uso della crittografia:

1. Il client chiede il certificato digitale del server;
2. La signing authority è letta nel certificato;
3. Se la signing authority è conosciuta ed è pre-installata nel browser, allora controlla la firma digitale, altrimenti la sicurezza NON è garantita;
4. Negoziazione sui parametri di sicurezza;
5. La connessione può essere stabilita

N.B: In HTTPS tutto è criptato!
Per ottenere certificati è necessario contattare la signing authority.

3.4 Sessione

HTTP è un protocollo stateless, ossia anonimo e ogni richiesta è indipendente dalle altre.
Per tenere traccia di ogni client lungo diverse richieste **vengono associate dei dati allo specifico client** che prendono il nome di **sessione.**

N.B: Authentication != riconoscimento sessione

COOKIES : Una semplice tecnica per riconoscere il client, attraverso le diverse richieste, è memorizzare nel web browser coppie chiave-valore, che il server chiede al client, per futuri utilizzi. Esse vengono inviate negli HTTP headers e gestiti dal browser(**sono invisibili all'utente**).
N.B: I cookies sono importanti per il server, ma non così importanti per il client.

3.4.1 Tipi di cookies

- Session cookies : Dipendono dal life-cycle del browser;
- Persistent cookies : Sono salvati con sicurezza nel file-system e il server specifica chiaramente la loro durata. Possono avere anche una lunga durata e potrebbero non essere eliminati alla chiusura del browser;
- Secure cookies: Usati solo con HTTPS quando vengono inviati nuovamente al server dal client;
- HTTPOnly cookies: Non possono essere letti da script Javascript (prevengono l'attacco Cross-site scripting (XSS));
- Samesite cookies: Cookies che possono essere inviati solo se un server A ha fornito la risorsa contenente un link a un'altra risorsa fornita sempre dal server A. Cerca di mitigare il problema del Cross-site request forgery

3.4.2 Cross-site scripting (XSS)

Il cross-site scripting è una vulnerabilità che consiste nel inserire codice Javascript malevolo per poi eseguirlo in quella pagina web.
Se si è vulnerabili ad attacchi XSS, a volte usare i cookie HTTPOnly non funziona, perchè un altro tipo di vulnerabilità possibile è quella di usare un keylogger così da sapere cosa sta digitando l'utente sulla tastiera. Prevenire quindi un attacco XSS è molto importante.

3.4.3 Cross-site request forgery (CSRF)

Il **cross-site request forgery** è un problema di sicurezza, che consiste in far passare le richieste di un utente autenticato a siti malevoli perchè il browser non sa se le richieste sono di quel utente o fatte da uno script.

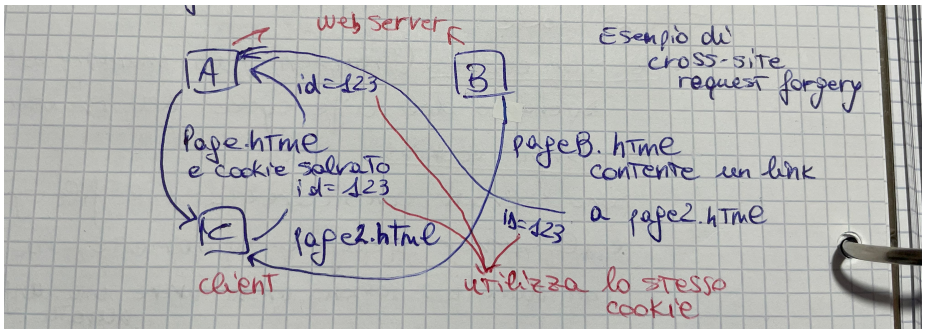


Figure 1: Esempio di come funziona CSRF

Ulitizzando il samesite cookie il problema di utilizzare lo stesso cookie per diverse richieste non accade.

Un tipico scenario in cui ciò può accedere è nella web app della banca in cui un trasferimento di denaro viene effettuato anche se non ho dato il consenso perchè viene utilizzato lo stesso cookie, sempre se questo non ha superato la data di scadenza.

3.4.4 Cookie-header

La struttura di un HTTP response header che un server deve sempre inviare a qualunque richiesta è la seguente:
$Set-cookie : name = value[;expires = date][;path = path][;domain = domain][;secure][;HTTPOnly][;Samesite]$

- name = value : indica la coppia chiave valore del cookie;
- expire=date : indica la durata massima del cookie;
- path = path : indica il percorso che deve esserci nell'URL richiesto, affinchè il browser per invii l'header del cookie;
- domain=domain : indica l'host in cui inviare il cookie
- secure : indica che il cookie viene inviato solo se la richiesta è effettuata con HTTPS;
- HTTPOnly : Nega l'accesso ai cookie tramite script Javascript
- samesite : Controlla se il cookie è inviato o meno da richiste cross-site.

3.4.5 Cookie, security e privacy

I cookie, inizialmente non hanno rappresentato un problema, perchè il client poteva scegliere di non inviare al server il cookie che gli era stato chiesto di salvare nel browser. Tuttavia **per alcune applicazioni però è necessario l'utilizzo dei cookie.**
Una pagina web spesso contiene cookie di terze parti, ad esempio i banner pubblicitari provenienti da altri web server, che possono tenere traccia dei nostri movimenti sul web, creando così un problema per la nostra privacy.
Per tali motivi **in Europa vi è un regolamento specifico per questi tipi di cookie, in cui è obbligatorio per il client dare il proprio consenso all'utilizzo di questi cookie.** Altri problemi di sicurezza sono ad esempio il Network eavesdropping nel quale se si utilizza il protocollo HTTP chiunque potrebbe leggere il traffico che passa su quella connessione. Ecco il perchè viene utilizzato molto HTTPS in cui tutto è criptato.

N.B: I cookie non identificano con precisione un utente, perchè essi dipendono da una tripla composta da: browser, computer e account. Basta cambiare uno di questi componenti e il cookie non ci identifica più in maniera univoca, generando così problemi di autenticazione tra device multipli.

4 Autenticazione

HTTP supporta vari meccanismi per autenticare un client fornendo le sue credenziali.

DEF: **Autenticazione vuol dire mostrare alcune evidenze dell'identità fisica di un particolare client.**
Solitamente è basato su informazioni condivise tra server e client come username e password, inserite nel body della richiesta.(Queste info non devono essere conosciute da altre persone, almeno la password.)

HTTP supporta 2 tipi di meccanismi di autenticazione:

- **BASIC ACCESS AUTHENTICATION**
- **DIGEST AUTHENTICATION** vulnerabile ad attacchi Man-in-the-Middle

4.1 Basic Access Authentication

Basic Access Authentication è la forma di autenticazione più usata in caso di protocolli HTTPS, perchè con HTTPS tutto viene criptato attraverso certificati digitali.
Se richiedo una risorsa protetta, il server può rispondere con un errore 401 login required.
Oltre al codice 401, viene inserito nella risposta l'header WWW-authenticate che informa il client quali sono i dati di cui il server ha bisogno.
Si tratta di un **protocollo Challenge-Response** dove la challenge viene fatta dal client e la response dal server.

Come lavora il basic access authentication?

- Un **server** può **rifiutare una transazione**, fa un challenging al client con una richiesta di username e password.
- Se le **credenziali** sono **corrette** il server **fornisce** la **risorsa**
- Se le **credenziali non sono corrette** la **risorsa non viene fornita**

Gli step necessari sono:

1. Il client **invia** una richiesta (GET,POST,PUT,DELETE...)
2. Quando il client **riceve** la **risposta con WWW-authenticate nell'header** vuol dire che **deve inserire le credenziali perchè la risorsa è protetta.**
3. Le credenziali vengono inviate nel seguente formato $< username >:< password >$. Le stringhe di username e password poi vengono **concatenate** e viene fatto l'**encoding** delle stesse usando l'algoritmo **Base64**.

4. Successivamente tutte le richieste inseriranno nell'header $< Authorization >$: $Basic < crd >$. Il server così decodifica la stringa codificata precedentemente e ne controlla le credenziali. La risorsa, infine, viene inviata normalmente se le credenziali sono corrette.

Base64 codifica uno stream di byte in una stringa che può contenere solo caratteri alfanumerici. **NON E' UNA FORMA DI SICUREZZA perchè Base64 è invertibile.**

N.B: Usare sempre HTTPS in caso di autenticazione di un utente, proprio perchè Base64 non fornisce sicurezza.

4.2 JSON Web Token (JWT)

I **cookie** sono usati solamente come **identificatori** per un particolare client, ed esso **può essere modificato dal client.**
Il meccanismo tradizionale dove il client ha solo un ID e il server fa tutto il resto ha impatti negativi come:

- **Ridotta scabilità**: Un server deve mantenere i dati in memoria, per ogni user autenticato, usando quindi molta più memoria di quella necessaria.
- **Increased coupling between server e client:** la logica viene implementata principalmente dal server(il client quindi può fare qualche azione?)

Le **applicazioni SPA** (Single Page Application) hanno **l'obiettivo di minimizzare le interazioni con il server** spostando la logica sul client più che sul server.
Usando così molta meno memoria lato server, esso quindi **deve solo controllare la validità delle operazioni**.

Un modo efficiente per gestire tutto questo è usare quelli che si chiamano:
TOKEN.
Come funzionano?

- Il **server genera** un **token** all'**autenticazione** che contiene tutti i dati di sessione
- Il **server** inoltre **firma il token per assicurarsi che il client non possa modificarne il contenuto.**

Il **JSON Web token è una stringa codificata in Base64** divisa in 3 parti, ciascuna parte separata da un "."

- Un **header specifica** il **tipo** e la **firma** usata nel token
- Un **contenuto** che **contiene** i **dati** in **formato JSON**.
- **Firma digitale** di entrambi header e contenuto

Usare il token permette al server di non memorizzare più nulla e i dati non sono criprtati, ma firmati quindi non è possibile modificarli senza invalidare la firma. In altre parole il token non è più valido.

Ad ogni successiva richiesta, il token viene inviato al server come fosse un cookie con la seguente autorizzazione:$Authorization : Bearer < token >$ e copiato nel body di ogni richiesta.

VANTAGGI del JWT:

- Il server può recuperare la sessione dell'utente senza memorizzare nulla e senza ulteriori query al database. Quindi una maggiore scalabilità
- **Il token può essere scambiato tra domini diversi: Single-Sign-On.**
  Esso funziona nel seguente modo:
  Un utente può essere autenticato ad un dominio A per ricevere il JSON Web token
  Questo token può essere inviato al dominio B che verifica se il token è generato da A.
  Il dominio B opera considerando l'utente come autenticato perchè l'autenticazione è gestita dal dominio A.
- Il token **può essere condiviso** anche tra **web app e native application** senza alcun problema.

DOVE MEMORIZZARE IL TOKEN?
Esistono 2 possibili modi:

- **Browser local storage** : memoria fornita dal browser dove memorizzare ciò che si vuole localmente.
  PRO: molta memoria disponibile
  PRO: immune ad attacchi CSRF (perchè il token non viene inviato al server)
  CONTRO: vulnerabile ad attacchi XSS (perchè può essere accessibile da codice Javascript e quindi può essere "rubato")
- **In un cookie** (inserito nel suo header):
  PRO: immune ad attacchi XSS (se il cookie è HTTPOnly)
  CON: vulnerabile a CSRF CON: dimensione massima 4KB

Con quest'ultima tecnica perdo alcuni vantaggi dei token.
N.B: Obiettivo N°1 evitare attacchi XSS

4.3 JSON

JSON è l'acronimo di **JavaScript Object Notation** è un formato in cui fare l'**encoding** dei **dati**, detta **serializzazione.**

Questo formato è molto usato perchè ha la stessa sintassi di un oggetto JavaScript e quindi molto facile da convertire. Inoltre è lightweight e facile da leggere per le persone.

4.3.1 JSON Objects

Gli oggetti JSON hanno la seguente sintassi:

```
{"name" : "John" , "age" : 30, "car" : null}
```

Ogni property di un oggetto JSON è composta da coppe chiave-valore.

4.3.2 JSON arrays

Gli array JSON hanno la seguente sintassi:

```
["Ford", "BMW", "Fiat"]
```

Essi possono anche essere inseriti all'interno di oggetti JSON, ad esempio

```
{
    "name" : "John",
    "age" : 30,
    "car" : ["Ford", "BMW", "Fiat"]
}
```

5 Web Socket

Web Socket è un **protocollo** che **consente** una **comunicazione full-duplex**, usando una connessione TCP/IP.
E' un protocollo pensato per essere **compatibile con HTTP**, perchè inizia con un handshake basato su HTTP per poi fare uno switching in web socket.
Spesso tra client e server c'è un intermediario detto **proxy**. I motivi per cui si usano i proxy sono:

- In una rete ci sono molti server e il proxy decide quale server è più adatto a ricevere una determinata richiesta;
- I server proxy possono fare anche da web cache, e fornire immediatamente le informazioni necessarie, se già richieste poco tempo prima.

Una volta che la connessione web socket si è stabilita, **si possono inviare in qualsiasi momento messaggi tra client e server anche in contemporanea**, cosa che in HTTP non è ammessa.

Web socket ha 2 componenti:

1. **Handshake protocol**, basato su HTTP, serve per negoziare i parametri della connessione e stabilire la connessione stessa.
2. Un **meccanismo di frame** per il trasferimento dei dati con basso overhead e bassa latenza.

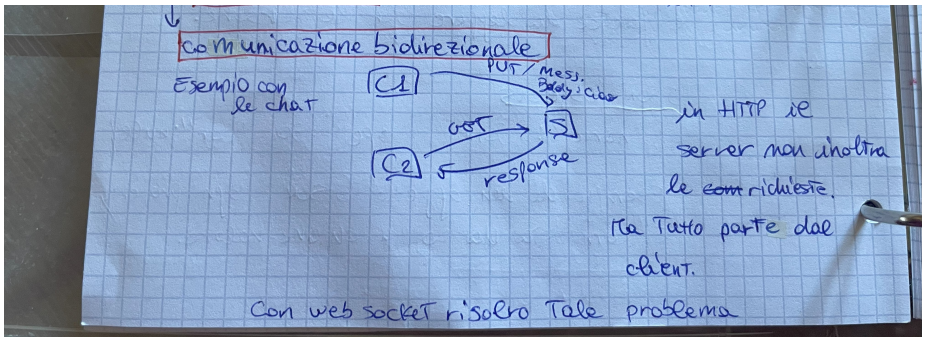


Figure 2: Esempio di come funziona una chat su una web app

5.1 Web Socket Handshake

La comunicazione viene stabilita su una connessione HTTP già esistente.
Il client chiede un **upgrade header** dell'HTTP usato per negoziare un protocol switch, inoltre è pensato per prevenire attacchi malevoli.
La risposta allo switching viene indicata tramite lo status code "101" se essa ha successo.
Ad ogni richiesta di protocol switching **la chiave viene hashata e salvata come cache per motivi di sicurezza.**
Dopo lo scambio di messaggi request-response:

- La **connessione TCP rimane aperta**
- **Nuovi messaggi sono scambiati usando il protocollo Web Socket**. In altre parole da qui in poi si utilizza solo Web Socket

N.B: Una volta che si passa a Web Socket non c'è modo di tornare a HTTP
I messaggi web socket sono **messaggi binari**, ossia uno stream di byte.
La comunicazione essendo full-duplex sia client che server inviano messaggi al bisogno, ed essi sono divisi in frame, inviati sequenzialmente e riassemblati a destinazione.
Di seguito alcuni campi che fanno parte del frame:

- Bit : può avere valore 0 o 1. 0 = altri frame necessari per completare il messaggio; 1 = messaggio completo
- Opcode : 3 bit che specificano il tipo di messaggio. I più importanti sono: (1) text, (2) binary (8) close (9) ping (10) pong
- Mask : Indica se il messaggio ha una "maschera"
- Length : indica la lunghezza dei dati da inviare.

Il contenuto del frame viene **offuscato** prima dell'invio con una **masking key** (ottenuta tramite operazione XOR) per evitare attacchi di tipo "cache poisoning".

6 REST APIs

Prima di parlare di API occorre specificare cosa s'intende con interfacce: esse definiscono quali dati e come vengono trasmessi tra i vari pezzi del software.
L' API (Application Programming Interface) definisce i metodi di comunicazione e i protocolli utilizzati tra i componenti di un software. Esse possono essere usate tramite il web prendendo il nome di WEB API.
Sono usate anche in contesti come le web app dove client e server comunicano proprio tramite API.
Nel momento in cui si sviluppano le API possono subentrare alcuni problemi come:

- Come fare encoding dei dati?
- Come invocare le funzioni presenti nell'API?
- Come dare un'interfaccia di alto livello per tutti i dispositivi che la utilizzeranno (Device independent)?

Per risolvere questi problemi si possono utilizzare varie tecnologie come le **REST API.**
Nella storia c'è stata anche un altro tipo di tecnologia alternativa ovvero le SOAP API. Queste API erano pensate per lo sviluppo dei web services, con un proprio linguaggio (WSDL) che definiva formalemente la loro interfaccia. Inoltre consentiva di generare automaticamente il codice delle varie richieste direttamente lato client. Però oggi non è molto usata.

REST: Reprensentational State Transfer. **Non è nè un protocollo nè uno standard, ma è un architectural style**, ossia uno stile che definisce delle linee guida su come implementare un'interfaccia API.

N.B: La tecnologia REST è basata interamente sul protocollo HTTP
Quindi segue il modello client-server in cui:

- Il server fornisce dei servizi
- Il client usa questi servizi tramite un canale di comunicazione

Idealmente quando il client chiede qualcosa al server tale richiesta deve essere indipendente dalle altre.
Quali sono i vantaggi delle REST API?

- **Scalabilità:** Il server è in grado di inviare più velocemente le informazioni senza salvarne altre al suo interno.
- **Reliability:** Un sistema stateless può recuperare facilmente da eventuali failure.
- **Facile implementazione.**

- **Visibilità:** Diventa facile monitorare il sistema.

Un'altra delle linee guida di una REST API è avere **un'interfaccia uniforme**, ossia fornire un'interfaccia uniforme **a tutti i possibili client**.

Un'architettura REST è pensata per lavorare bene in una rete con traffico sostenuto, perciò ogni componente è diviso in **layer** per semplificare la complessità e mantenere il controllo sui vari componenti.

6.1 Risorse

Il punto fondamentale delle REST API sono le **risorse**.
Esse rappresentano:

- Cosa farà il server
- Il tipo di info che fornirà il server
- Un'astrazione di tutte queste rappresentazioni può essere rappresentata nelle app.

Esse inoltre hanno:

- Un **URL**: chiamato anche **endpoint**
- **Una o più rappresentazioni della stessa risorsa** (più presentazioni diverse per quella determinata risorsa)
- Alcuni **metadati** per specificare info aggiuntive (es: data ultima modifica)
- **Control data:** Specifica alcuni constraint e direttive delle risorse

6.1.1 Resource identifier

Dovrebbe fornire un'**identificazione univoca** e il **percorso assoluto** (serve per dare un'organizzazione ai vari dati) della risorsa.

N.B: URL deve identificare la risorsa in qualsiasi momento e in qualsiasi punto del software.

REST prevede inoltre delle **azioni** che vengono **mappate in metodi HTTP** (GET, POST, PUT, DELETE ecc..) e contenere nell'URL dei parametri inseriti nella sezione delle query per le varie ricerche all'interno per esempio di un database.

6.2 API Design

Di seguito vengono fornite delle linee guida riguardo al design delle REST API:

1. Modellazione del database per uno specifico contesto

2. Definire che tipo di azioni possono essere eseguite dalla richiesta come operazioni CRUD, Autorizzazioni, Opzioni e attributi richiesti

3. Decidere gli endpoint

4. Definire i metadati associati (non cruciali) descrivono le risorse gestite dell'app

Le API usano delle **naming conventions** facilemente intuibili anche senza leggere la documentazione. Alcune di queste sono:

Gli **endpoint** devono avere nomi specifici e intuitivi.

URL devono essere riferiti alle risorse e NON all'azione. Questo perchè ho molti endpoint da creare ed è poco efficiente e non si sa che nome attribuire ai singoli endpoint.

L'encoding dei dati dovrebbe usare formati popolari come **JSON** o **XML**.

Le API subiranno **frequenti release** e alcune feature potrebbero cambiare oppure si possono aggiungere nuove tecnologie e funzionalita, perciò una buona idea è **indicare la versione** dell'API **direttamente** sull'**URL**, così da supportare più versioni della stessa API (es: /api/v1/...) v1 indica la major version.

6.2.1 Semantic versioning

Serve a fornire ulteriori informazioni sulla versione dell'API. Essa viene indicata da **3 numeri.**
Questi 3 numeri sono scritti nel seguente modo (es: 2.0.0)

1. **Major version**: quando vengono fatti dei cambiamenti nell'API tali da rimuovere la backward compatibility con le precedenti versioni. In altre parole tutti i clients devono adeguarsi alla nuova versione. Si incrementa il numero più a sinistra.

2. **Minor version**: aggiunge funzionalità nel backend ma rimane comunque compatibile con le vecchie versioni. Si incrementa il numero al centro.

3. **Patch**: versione in cui si eseguono dei bug fix. Si incrementa il numero più a destra

Un'ulteriore buona pratica nello sviluppo delle API è **generare una documentazione** che descriva come utilizzarla e come essa funziona.
Inoltre un'API in stile REST deve ritornare **messaggi di errore significativi** quando accadono.
Infine cosa importante è definirne la **sicurezza**, decidendo chi è autorizzato a utilizzala e anche la **scalabilità** quando la rete ha molto traffico perchè essa deve comunque funzionare.

7 Docker Containers

Per sviluppare una web app e poterla poi eseguire su più macchine senza problemi legati a framework e versione delle librerie, abbiamo a disposizione due possibili alternative:

- **Macchina virtuale**: contiene a tutti gli effetti un sistema operativo e tutte le risorse necessarie. Esse sono gestite dall'hypervisor che è quel componente della macchina virtuale in grado di gestire contemporaneamente diversi sistemi operativi.
- **Container**: sono macchine virutali lightweight la cui differenza con le macchine virtuali è che condividono lo stesso kernel del sistema operativo su cui sono ospitati e inoltre ogni container è isolato dagli altri.

I vantaggi di utilizzare i container sono:

- **Semplifica di molto il deployment del progetto**
- Ha uno **startup veloce** perchè non deve caricare le proprie risorse ma usa quelle già esistenti nella macchina in cui è installato
- **Portabilità**
- **Scalabilità**

7.1 Architettura docker

Docker utilizza un modello di comunicazione client-server in cui entrambi sono la stessa macchina fisica. Per poter interagire con il docker si può usare la CLI (Comand Line Interface) oppure la corrispondente GUI (Graphical User Interface).
Importatante nell'architettura del docker è il **Docker Registry**: esso è una repository di immagini supportate dal docker. Se al momento della creazione di un container essa non è già presente, verrà scaricata dalla repository ufficiale di docker. Qual è quindi la differenza tra immagini e container?

- **Immagini**: Sono dei template da cui si possono creare più container. Le immagini scaricate sono definite/descritte all'interno di un file chiamato **Dockerfile.**
- **Container**: sono delle istanze eseguibili di un'immagine. A questi può essere dato un nome.

Quindi facendo un parallelo con la programmazione le immagini sono come le classi, mentre i container sono gli oggetti instanziati da tali classi.

7.2 Creazione di un'immagine e di un container in docker

Di seguito vengono riportati alcuni comandi per scaricare le immagini e creare da esse dei container:

- **docker run <nome immagine>** verrà scaricata l'immagine indicata e creato un container su di essa. Il nome del container verrà scelto casualmente
- **docker run -it -d –name ubuntu** scarica l'immagine di ubuntu in background(-d) e dico di voler interagire(-it) con essa (ad esempio dalla shell).
- **docker exec -it nomeContainer /bin/bash** eseguo uno specifico programma presente nel container creato.

N.B: I **nomi** delle **immagini** sono **definite** da **chi le ha create**(possono essere cercate su docker hub per conoscerne il nome); i **nomi** dei **container** sono messi random al momento della creazione ma **si possono personalizzare**.

7.3 Single-Host Networking

Con Docker è possibile condividere le risorse tra container e host mappando le porte interne del container con quelle dell'host su cui è installato, così da poter condividere gli stessi file:

docker run -d –name <nomeContainer> -p 8080:80 nginx : alpine

N.B. Il comando *-p* indica che ogni processo in esecuzione sulla porta 80 del container è condiviso sulla porta 8080 dell'host.

7.4 File system dei container

Un'immagine è un file zip contente, oltre al template per creare i container, anche un **layered file system**.
Perchè layered file system:

- In ogni layer ci sono cartelle e file
- Ogni layer contiene i cambiamenti rispetto ai layer sottostanti
- Quando si crea un container il suo file system è l'unione di tutti i layer e di conseguenza di tutte le modifiche effettuate.
- Ogni layer è READONLY nelle immagini, nei container è stessa cosa eccetto per il top-layer che invece è l'unico modificabile.

N.B: I layer NON MODIFICABILI vengono referenziati dall'immagine da cui sono stati creati i container. (NON COPIATI PER OGNI CONTAINER)

7.5 Copy-on-write

In caso di modifiche il file modificato viene prima copiato sul top-layer e successivamente modificato.
Con il comando

docker container diff <container>

Mostra i file modificati nel container

docker container commit <container> <image>

Rende permanenti le modifiche dei file creando un nuove immagini a partire da quei file modificati. Questo processo può essere "scriptato" nel file denominato Dockerfile.
N.B: Tutti i container nel top-layer contengono tutti i file modificati nel file system durante la loro esecuzione.

7.6 Volumi

E' possibile copiare file dentro e fuori dal container con il seguente comando:

docker cp <container>:SRC DST

Consente di copiare il file fornendo i percorsi di sorgente e destinazione. (Entrambi percorsi presenti nel container)

docker cp SRC <container> : DST

Consente di copiare i file fornendo come SRC il percorso del file presente sull'host e come destinazione un percorso presente nel container.

Inoltre esistono due modi per condividere dati e mantenerli per poter usarli con diversi container:

- **Volumi** : Hard-disk virtuali possono essere usati SOLO tra CONTAINER garantisce migliori performance
- **Bind Mount**: Drive condiviso tra container e host.

Comando per creare volumi

docker volume create myvolume
docker run -v myvolume:<mountpoint> <image>

N.B: I dati rimangono salvati anche se si esce dal container oppure anche in caso di eliminazione perchè sono salvati su un hard disk esterno al container.

docker run -it –name <container> -v <path del file da condividere>

Non molto veloce, inoltre sto spostando parte del file system in un container (problema per la sicurezza)

7.7 Creare e gestire immagini

Il processo di creazione e gestione delle immagini può essere automatizzato nel **Dockerfile**.
Il **Dockerfile** è un text file che consente di fornire istruzioni per costruire immagini custom. Il tutto usando il comando:

docker image build -t <image>

Come anticipato prima è possibile creare delle reti affinchè più container possano comunicare tra loro con il seguente comando:

docker network create <network name>
docker run –name c1 -it –rm –network <nome> <image>

Consente di creare una rete e aggiungere il container c1 alla rete creata.

8 Javascript

E' un linguaggio **single-thread** e ogni **funzione lavora** in **maniera asincrona**.
Usa il paradigma della programmazione ad oggetti attraverso l'uso dei **prototypes** e fa un grande uso della programmazione funzionale.
Esso è il dialetto più famoso ed esistente ad oggi di **ECMAscript**. ECMAscript è uno **standard** che ha numerose versioni e a partire alla versione 5 è supportato da tutti i browser.
Inoltre Javascript è un linguaggio ad **alto livello**, **interpretato** (c'è un interprete che esegue il codice riga per riga), **untyped** (il tipo non è associato all'identificatore(nome) della variabile, ma al suo valore) e **dinamico** (non è sempre facile sapere quale funzione è in esecuzione).

8.1 Dettagli di Javascript

Per eseguire un programma Javascript c'è bisogno di un **interprete**, esso esegue il **parsing del file riga per riga** e la esegue al volo.
I **linguaggi interpretati** sono **più lenti** di quelli compilati, anche se ad oggi Javascript supporta la tecnologia **JIT** (Just In Time), ossia se **un'istruzione viene eseguita più volte, la prima volta viene interpretata e "cachata" in modo da essere più veloce nelle successive esecuzioni**.
Dinamico significa che si può **modificare il programma anche quando è**

in esecuzione. Altro significato di dinamico è **poter usare la stessa variabile con diversi tipi di dato.**
N.B: NON C'E' UN TYPE CHECKER.

Infatti Javascript tenta di fare qualsiasi operazione anche usando CONVERSIONI IMPLICITE. E' comunque possibile limitare questi cast usando la modalità "**strict**" esistono inoltre linguaggi come Typescript che usano un type system per limitare tale problema.

Le librerie di Javascript nativo sono pochissime (solo quelle per la gestione dei dati), infatti la maggior parte sono librerie di terze parti che delegano tanto all'ambiente in cui sono installate.

Javascript supporta la programmazione OOP, ma in maniera molto diversa da linguaggi come Java,C++, infatti **non esiste un concetto di classe** (sebbene esista la keyword class),**bensì di oggetto**.
Il meccanismo di ereditarietà è basato appunto sugli oggetti, non sulle classi. Esiste un meccanismo di **oggetti collegati ad altri oggetti**. Esso consiste nel fatto che **ogni oggetto contiene un pointer ad un altro oggetto detto PROTOTYPE.**

Permette l'uso della programmazione funzionale. Le **funzioni** infatti per Javascript sono importanti e inoltre **sono anche degli oggetti**, **con un loro prototype.** Posso crearle al volo e usare le lambda, supportano le **closure** ossia **possono catturare le variabili definite fuori dal contesto della funzione.**

8.2 Tipi di dato

Tipi primitivi:

- Number vale per tutti i numeri (interpretati come double)
- String
- Boolean
- null
- undefined (indica un valore di default non definito)

Tipi reference

- Tutto ciò che non è tipo primitivo, ossia tutti gli oggetti

8.3 Oggetti in Javascript

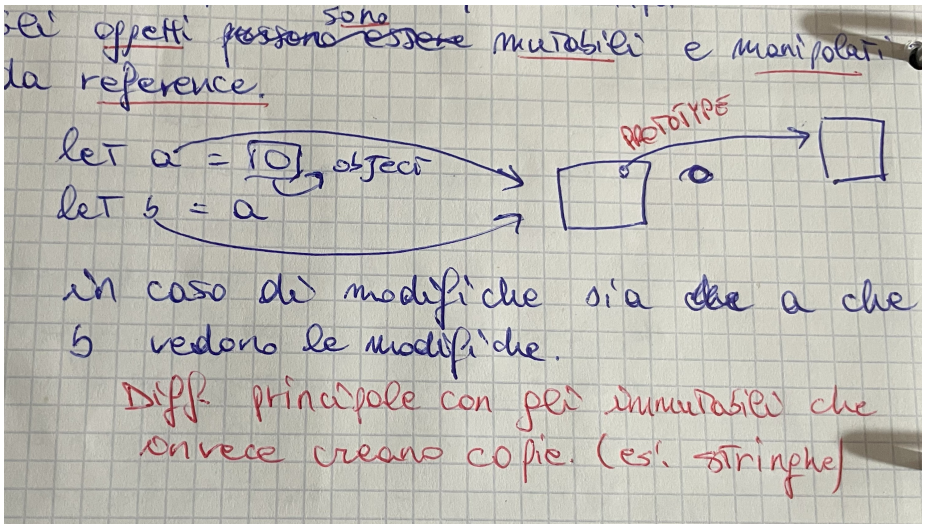
Sono una **collezione di properties** e ogni property è una **coppia chiave-valore**.
Anche le funzioni hanno delle properties a cui è associato codice da eseguire che è possibile invocare. I tipi possono essere:

- **mutabili** : posso cambiarne il valore associato
- **immutabili** : non posso cambiarne il valore associato (readonly)

Tutti gli oggetti sono **mutabili**, mentre tutti i tipi primitivi sono **immutabili.**
In questo caso **prima di ogni modifica** viene **creata** una **copia** di tale **variabile**.

8.3.1 Oggetti nel dettaglio

In Javascript come già detto gli oggetti sono una collezione di property il cui **nome** è una **stringa** e il **valore** può essere di **qualsiasi tipo.**
Gli oggetto vengono manipolati tramite reference.



Ad **ogni oggetto è associato un prototype**, indicato con il nome di __proto__ e in certi casi **il prototype può puntare a null**.
Questi puntatori ad un oggetto creano una linked list che prende il nome di **catena di prototype** e danno vita al concetto di **object linked to other object**.

8.3.2 Accesso alle property

Per accedere alle property dell'oggetto ho 2 modi possibili:

- **user.name** : in questo caso la property name deve essere presente nell'oggetto che ha come nome user, ossia deve essere conosciuta a priori.
- **user["name"]** : in questo caso posso anche inserire variabili all'interno delle [], in altre parole ciò che scrivo tra le parentesi [] può essere conosciuto a runtime, inoltre generalmente le properties di un oggetto sono visibili a tutti. Posso usare questa sintassi perchè l'oggetto è una collezione di properties.

N.B: **Viene valutata per primo ciò che è scritto a sinistra dell'operatore "." e non deve essere null. Tuttavia da ES2020 esiste l'operatore ?. che ritorna undefined se l'oggetto non esiste.**
Ad esempio:

```
let o = {}; // empty object
o.a = 3; // crea una property in "o" con valore 3
o2.a; // "o2" non esiste quindi dar  un'eccezione
o2?.a // torna undefined
```

8.3.3 Global object

Si tratta di un oggetto "speciale" allocato all'inizio dell'interprete e **contiene tutte le variabili globali e funzioni usabili ovunque**.
In base all'ambiente in cui viene eseguito lo script, il global object può assumere diversi nomi:

- Browser : **window**
- Worker : **WorkerGlobalWorker** consente di eseguire thread multipli in parallelo. Utile per vedere dove viene eseguito lo script
- Node.js : **global**

8.3.4 Creazione di un oggetto

Ho anche qui 3 modi diversi:

1. **Letterale oggetto**:

   ```
   let book = {title : "Javascript", pages : 200};
   ```

   Crea un oggetto al volo specificandone il contenuto e lo assegno ad una variabile. Posso aggiungere altre property oppure eliminarne.

2. **Uso della keyword new seguita dall'invocazione di una funzione**:

   ```
   let a = new Array(); let b = new Date();
   ```

In questo caso Array() e Date() prendono il nome di **costruttore**.

3. **Usando Object.create(<prototype>)**

```
let o = Object.create({x : 10, y : 20});
```

Torna un nuovo oggetto il cui prototype è quello indicato nel metodo create(), quindi specificato manualmente.

N.B: In generale la catena di prototype non è modificabile.

Qual è la differenza tra questi 3 modi di dichiarare un oggetto?
Tutti gli oggetti creati come **letterali oggetto** hanno come **prototype Object.prototype.**
Se uso la keyword **new** il **prototype del nuovo oggetto sarà il protoype del costruttore**, perchè sebbene sia una funzione per Javascript anche le funzioni sono oggetti.

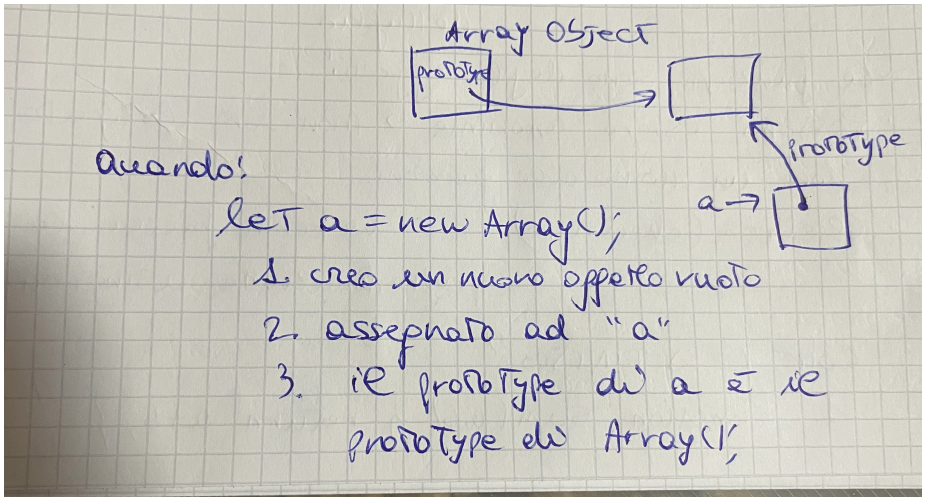


Figure 3: Fasi della costruzione di un oggetto

In caso di **lettura** del valore di una property, **viene percorsa la catena di prototype** alla ricerca di tale property per stamparne il valore.
In caso di **scrittura** di una property essa **viene aggiunta nell'oggetto corrente, senza considerare un'eventuale presenza di quella property lungo la catena.**

8.3.5 Delete property

Per eliminare una property da un oggetto si può usare l'operatore **delete** e cancella SOLO nell'oggetto passato all'operatore. NON viene PERCORSA la catena di prototype.

```
delete book.author; // cancella la property author dell'oggetto book
```

8.3.6 Spread operator

Lo spread operator è **"..."** serve per copiare il contenuto di un oggetto in uno nuovo.
N.B: Il suo utilizzo è possibile SOLO nei LETTERALI OGGETTO e non modifico il prototype.

8.4 Funzioni di Javascript

Le funzioni sono degli oggetti! Possono essere anonime, ossia non sono obbligato a dare un nome alle funzioni. **Esistono le clousure (chiusure)** servono a rendere più semplice la programmazione asincrona.

8.4.1 Definizione di funzione

Ho 3 modi per definire una funzione:

1. ```
   function sum(a,b) {return a+b;}
   ```

2. **Function expression:**

   ```
   let sum = function(a,b) {return a+b;}
   ```

   Dichiaro una funzione anonima e assegno il suo valore ad una variabile. Per riferirmi a quella funzione utilizzo il nome della variabile.

3. **Funzioni freccia (da ES6):**

   ```
   let sum = (a,b) => {return a+b;}
   ```

   Stesso concetto della function expression, ma uso uno shortcut per creare la funzione specificando i parametri, la freccia e il corpo della funzione.

La principale differenza tra il secondo e il terzo modo è come lavora la variabile **this**. In entrambi i casi sto definendo funzioni anonime e assegnando il valore ad una variabile.
La differenza tra il primo e il secondo modo è che nel **primo modo NON posso usare la funzione in un if-statment** e soprattutto è **hoisted**,ossia la funzione viene copiata e incollata all'inzio dello script ed eseguita solo quando viene invocata. **Il secondo modo invece rende la funzione eseguibile in un if-statement, ma non avviene l'hoisting**; viene invece eseguita subito dall'interprete.
   ```

8.4.2 Invocazione di una funzione

Per invocare una funzione si ha a disposizione 4 modi:

1. **Invocation expression**: è l'invocazione classica f(a);
2. **Invocazione come metodo** (method invocation): se la funzione è definita come property di un oggetto.

```
let o = {f : (a) => {return a*10;}}
o.f(3); // invocazione della funzione come metodo
o["f"](3); // uguale a sopra
let f2 = o.f;
f2(3); // invocazione come funzione
```

3. **Invocazione come costruttore**: se preceduta dalla keyword **new**

```
let o = new Object(); // esempio di invocazione
                      // come costruttore
let k = new f2(4); // crea un oggetto vuoto k
                   // il cui prototype f2
```

4. **Indirettamente tramite funzioni come call() e apply()**: call() accetta una lista di parametri e posso definire il contesto della funzione; apply() accetta un array come parametro.

Da ES2020 posso usare l'operatore **?.** esempio f?.(), per controllare che f esista.

8.4.3 Keyword this

La keyword **this** si **riferisce al contesto** (oggetto) **in cui un metodo viene eseguito.** In base al contesto this assume diversi valori tra cui:

- **Se** sono **fuori dal blocco** in cui è definita la funzione, **this** si **riferisce** al **global object**.
- **Se** sono **dentro** ad una **funzione invocata come function expression this** è **undefined se** viene usata la **strict mode** (può essere attivata scrivendo "use strict" all'inizio della funzione), **altrimenti** si riferisce al **global object.**
- Se la funzione è **invocata come metodo**, **this** si **riferisce all'oggetto su cui la funzione viene invocata.**
- Se definita nella **catena di prototype**, **this** si riferisce comunque **all'oggetto in cui è invocata**.

8.4.4 Funzioni arrow

Non hanno la property prototype, e **non possono essere invocate come costruttori**. This è ereditata dall'ambiente in cui è definita e viene chiamato **lexical this.**

8.4.5 Variadic function

Sono **funzioni che accettano un numero arbitrario di parametri**. In Javascript questo è supportato **impostando** le **variabili**, a cui non sono assegnati i valori al momento dell'invocazione, come **undefined**. Se vengono passati più parametri di quelli previsti è possibile usare l'oggetto **"arguments"** per ottenere tali valori.

8.5 Chiusure (closure) in Javascript

In Javascript è possibile definire **funzioni dentro altre funzioni**. Le funzioni **più interne possono accedere** inoltre **alle variaibili definite fuori dal proprio scope**.

```
let scope = "global scope";
function checkscope(){
    let scope = "local scope";
    function f() {return scope;}
    return f();
}
checkscope(); // ritorna local scope come risultato
```

Se invece...

```
let scope = "global scope";
function checkscope(){
    let scope = "local scope";
    function f() {return scope;}
    return f;
}
checkscope()(); // ritorna comunque local scope
```

Nell'ultimo esempio il risultato che ritorna la funzione checkscopre()(), è lo stesso della precedente perchè ho le **chiusure**,**una volta catturata dalla funzione, tale variabile fa parte della funzione.**
Se una funzione non torna nulla ciò che vedrò come output quando invocata sarà undefined.

```
function counter(){
    let n = 0; // privata perch  rimane all'interno di counter()
    return {
        count : function(){return n++;},
        reset : function(){n = 0;}
    }
    let a = counter();
    a.reset(); // stampa undefined
}
```

8.6 Classi in Javascript

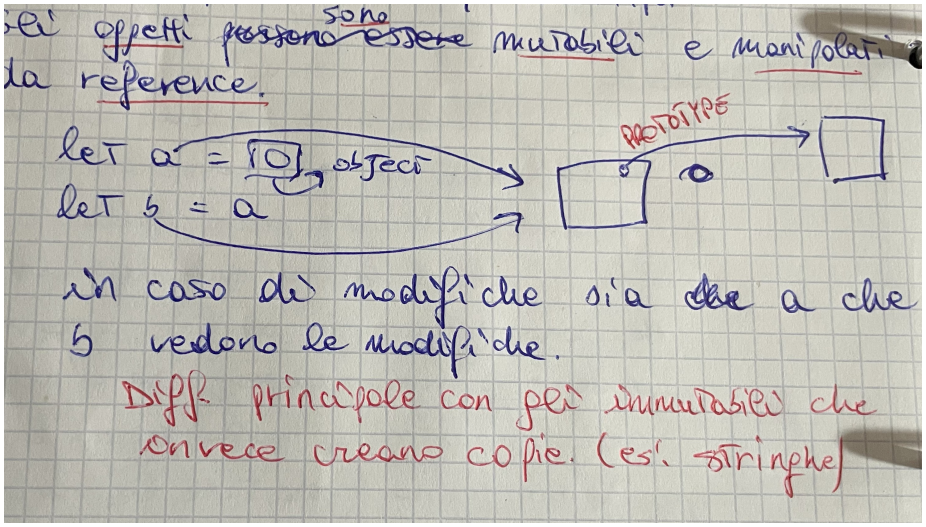
Le classi sono dei **blueprint** per gli oggetti che verranno creati da esse. In Javascript però l'**ereditarietà** si basa sui **prototype** non sulle classi perchè tale concetto non esiste.
N.B: Se due oggetti ereditano properties dallo stesso prototype, allora possiamo dire che essi sono istanze della stessa "classe".

8.6.1 Costruttori

Il modo classico per creare oggetti è usando i **costruttori**.
Quando si usa la keyword **new** il prototype del costruttore è usato per impostare il prototype del nuovo oggetto appena creato.
L'operatore **instanceof** consente di sapere se due oggetti appartengono allo stesso prototype.

8.6.2 Sottoclassi

É possibile estendere una classe **modificando il suo prototype**.
Se ad esempio B estende A, allora il prototype di B deve essere "figlio" del prototype di A. Per "figlio" s'intende che il prototype di B deve avere come prototype quello di A.




```
class RangeStep extends Range{
    constructor(from,to,step){
        super(from,to);
        this.step = step;
    }
}
```

L'interprete per ricreare *super(from,to);* utilizza la funzione apply(): *Range.apply(this,arg);*
Le **closure** sono **utili anche per rendere private le property di un oggetto** Javascript.

8.7 Stringhe in Javascript

Tutte le stringhe in Javascript usano la codifica UTF-16 e ogni carattere può essere acceduto tramite indice e le **stringhe sono iterabili**. **Non esiste un tipo per il singolo carattere.**

Da ECMAscript 6 è presente il **template Literal** che serve per **creare nuovi tipi di dato custom**, usando il **backtick**. Si possono inserire più linee per tale stringa e usando le parentesi graffe posso inserire il valore di una variabile nella stringa.
Qual è la differenza tra null e undefined?

- **null** è usato per indicare assenza normale o aspettata di un valore
- **undefined** è usato per indicare qualcosa di inaspettato a livello di assenza (es: variabile non inizializzata, variabile di un oggetto non presente)

N.B: Ogni variabile in Javascript può essere convertita in boolean. Tutto ciò che ha un valore è valutato come true, mentre tipi come null e undefined è false.

8.8 Statement

Sono **espressioni che producono side-effect**, ossia modificano la memoria. Sono degli statement:

- **assegnamenti**
- **incremento / decremento**
- **delete** (keyword per rimuovere una property da un oggetto)
- **invocazione di una funzione**
- **blocchi con le parentesi graffe**
- **espressioni condizionali**
- **loops**: for/while

Esistono 2 tipi di for

- **for...of** itera su un oggetto per ogni elemento presente
- **for...in** itera sulle chiavi di un oggetto iterabile. Possibile nel moderno Javascript

try-catch: lancia un'eccezione e la cattura. **N.B: Attenzione ad usare le eccezioni con le funzioni asincrone, per esse infatti esistono altre tecniche.**
Se e solo se il try block lancia un'eccezione allora viene eseguito il blocco catch. Altro statement è il **finally** che verrà sempre eseguito indipendentemente dal risultato del try-catch.

8.9 Variabili

Le variabili si possono dichiarare in 3 modi:

1. **let** : (es: let a;) in questo esempio il suo valore è undefined. Si possono anche dichiarare più variabili usando la "," oppure assegnarne subito un valore.

2. **const**: come suggerisce il nome questa variabile **non può essere modificata** e ci assegno un valore. Quando uso let e const le variabili sono **blocked scope** ossia sono visibili solamente nello scope in cui sono state create e nei nested scope. Se dichiarate fuori dallo scope sono **globali** e qui dipende dall'ambiente. In node.js sono definite in un file apposito, nel browser vengono definite nel file .html e se cambio pagina le perdo tutte.

3. **var**: Nelle vecchie versioni di Javascript si usava solo la keywork var. Il suo comportamento è un po' strano: **non esiste il block scope** e la dichiarazione è **hoisted**, ossia il **valore viene assegnato quando viene eseguita nel suo blocco di codice.** Se **dichiarata** all'**esterno** sto **aggiungendo** implicitamente una **property** al **global object**.

9 Typescript

Aggiunge un type system a Javascript e restringe alcuni comportamenti "strani" di Javascript.
É un **superset tipato di Javascript** che aggiunge alcune feature come un **compilatore** detto **transpiler** che controlla eventuali errori tra i tipi.
Typescript è quindi un linguaggio **compilato** che traduce il codice, in codice Javascript.Questa è già una prima differenza rispetto al Javascript puro.
Il compilatore di typescript è scritto in Javascript e può essere installato attraverso il pacchetto npm con il seguente comando:

npm install -g typescript

il compilatore typescript può essere invocato su un file .ts

tsc inputfile.ts

oppure, se installato localmente

npx tsc inputfile.ts

9.1 Javascript e typescript

Ogni codice Javascript è un valido codice Typescript, questo perchè typescript aggiunge un type system che è più restrittivo di Javascript e può generare dei warning anche se il file di input è sintatticamente un file Javascript valido.

9.2 Feature di Typescript

- **Type checker**: I tipi possono essere specificati staticamente e inoltre il type inference automatico sull'assegnamento delle variabili
- **Funzioni**: Contengono parametri tipati, parametri opzionali e di default ed è possibile eseguire l'overloading
- **Classi e interfacce**: Definizione esplicita di classi e interfacce, inoltre è presente l'ereditarietà tra le classi
- **Moduli**: I moduli di Typescript servono per definire namespaces, ossia crea dei piccoli file con definizioni. Inoltre consente la creazione di moduli esterni.

9.3 Type annotation

Anche se Typescript fa inferenza automatica(il compilatore riesce a determinare automaticamente il tipo di una variabile o espressione), può essere utile specificarli chiaramente tramite type annotation:

```
var <identificatore>:<annotazione> = <value>
```

9.4 Tipi base

I tipi base di Typescript sono:

- **Number**
- **Boolean**
- **String**
- **Array**< *base* > **o base[]**
- **any** : disabilita il type checking quando il tipo è sconosciuto durante la transpilazione (PERICOLOSO!!!!)
- **void** : Nessun valore

- **Object**: Tipo Javascript non primitivo

N.B: Anche se il compilatore Typescript rileva un errore, comunque viene prodotto un output perchè un file Javascript è anche un valido file Typescript.

- **Annotate functions:** < *type* >< *name* >:(name:< *intype* >) => < *outtype* >

```
let sayHello : (name: String) => String = (name) => {return name;};
```

- **Annotate objects:** < *type* >< *name* >:{name:String, height:Number};
- **Parametri opzionali:** ...<variable name>?.:<variable type>
- **Parametri default:** <variable name> = <value>

9.5 Interfacce

Sono usate in maniera molto simile ad altri linguaggi object-oriented. Il suo utilizzo è pensato per:

- definire metodi/attributi pubblici di una classe
- definire la struttura di un oggetto
- definire operazioni disponibili da librerie di terze parti

N.B: Non producono codice Javascript, ma sono utili solo per il type checker e possono estendere altre interfacce o altre classi.
Le interfacce sono open-ended, infatti è possibile aggiungere nuove properties all'originale.

9.5.1 Implementazione interfacce

```
class <name> implements <interface name>
```

Inoltre è stata introdotta una nuova keyword: **readonly** modificabile SOLO nel costruttore.

```
constructor(<visibility operator> <name>:<type>)
```

9.6 Sottotipi strutturali

Un tipo X è compatibile con un altro tipo Y se e solo se Y ha tutti gli attributi di X.

9.7 Type assertion

La sua sintassi è:

```
<type> variable or variable as <type>
```

Ha uno funzionamento simile al **casting**, ma limitato al contesto del type checker.Non viene fatta alcuna conversione a runtime.

10 NodeJS

NodeJS è un interprete Javascript che serve per eseguire codice Javascript.
Nello specifico è un **Javascript runtime**, molto simile all'interprete presente in Google eliminando tutto ciò che serve al browser e aggiunge un layer che dialoga con il sistema operativo.
É open-source, cross-platform ed è progettato per sviluppatori server-side.
Viene distribuito con:

- **REPL frontend**: ossia un interprete interattivo tramite command line
- Prevede un comando specifico per eseguirlo (**node**)
- **Un package manager npm** che serve per: gestire le installazioni di moduli aggiuntivi, risolvere le dipendenze tra moduli, installare e gestire command line tool aggiuntivi.

La sintassi per eseguire Node.js è:

node javascriptfile.js

L'entry point per l'interprete viene specificato in un specifico file chiamato **package.json**, in quanto un progetto può essere composto da molti file e cartelle. Inoltre definisce anche i metadati del progetto.
In alcuni casi tale file è auto-generato da un tool esterno.
Ecco un esempio di package.json

```
{
    "name" : "my project name",
    "main" : "main project.js",
    "version" : "0.0.1",
    "dependences" : {
        'colors' : "0.5.0"
    },
    "private" : true
}
```

"main": definisce l'entrypoint del progetto specficando il file Javascript da cui iniziare l'esecuzione.
"dependences": ciò che viene specificato deve essere installato nel file-system locale (quindi scaricato da npm)

10.1 Moduli

Un node package definisce un **modulo** che può essere eseguito o usato come libreria.
N.B: Ogni file Javascript è un modulo.

Il modo in cui vengono **gestiti i moduli non è standard**.NodeJS segue due possibili modalità:

- **CommonJS**: Questa è la modalità più usata che consiste nel utilizzo della funzione **require()**, specificando il nome del modulo. Essa consente di caricare moduli globali e locali. Nello specifico esegue il file e tutto ciò che è presente nell'export object viene esportato.
- **ES module syntax**: essa utilizza due keyword: **export** esporta le funzioni e **import** che importa le funzioni da altri file.

N.B: I 2 modi possono coesistere ed essere usati insieme. Se utilizzo il 2° modo l'estensione del file deve essere mjs invece di js.
In **Typescript** invece per caricare le funzioni si utilizza solamente la keyword **import** senza modificare alcuna estensione.

Differenza tra CommonJS e ES Module

- In **CommonJS** tutte le funzioni vengono **caricate** in **maniera sincrona**, l'interprete esegue il file importato e una volta terminato torna dove si era "bloccato". Si può usare anche lungo il codice.
- In **ES Module** il file viene caricato in **maniera asincrona** ed **eseguito** in **background**. Questo lo rende più veloce. Devo però specificare all'inzio del file quali file vengono importati.

10.2 npm

npm è il package manager da cui si può installare Node.js tramite il comando:

npm install

Esso infatti installa tutte le dipendenze localmente in una cartella denominata **node_modules** presente nella root del progetto.

Possiamo individuare 3 concetti chiave di Node.js

- **Operazioni asincrone**
- **Codice single-thread**
- **Concorrenza condivisa**

N.B: In Node.js c'è una sorta di scheduler che esegue ogni funzione in maniera asincrona, sebbene il codice di per sè sia single-thread.

10.2.1 Operazioni asincrone

Un modo per eseguire operazioni asincrone è il seguente:

```
async function(cb){
    ...
    return cb(a);
}

let i = f((i) => i = i+1);
```

Nel precedente esempio si utilizzano le **callback** ossia viene passata una funzione come parametro alla funzione indicata come asincrona che viene eseguita al termine, invece di ritornare direttamente il valore. In questo modo l'interprete può continuare ad eseguire le righe successive del file senza aspettare il risultato della funzione asincrona.

10.2.2 Codice single-thread

Node.js esegue il codice in maniera asincrona, ma non in parallelo lato Javascript.
Ha un singolo evento che esegue le callbacks. Internamente Node.js gestisce un thread pool non blocking-IO e ciclicamente passa attraverso diverse fasi. Inoltre alcune azioni che Node.js esegue vengono ripetute per ogni fase come ad esempio le MicroTask queue eseguite attraverso promises.

10.2.3 Concorrenza condivisa

Tutte le funzioni condividono lo stesso spazio di memoria, perchè essa è condivisa essendo il linguaggio single-thread. Non ho alcun overhead dato da creazione e distruzione dei thread.
In conclusione **Node.js è utile quando ho programmi I/O bound**, dove le operazioni input/output sono molto frequenti, come nel caso di web services e databases.
Non conveniente per programmi CPU-bound, dove c'è un uso intenso della CPU.

10.2.4 NodeJS APIs

Node.js fornisce un set di API utili per interfacciarsi con il sistema operativo tra cui quelli riguardanti HTTP/HTTPS.

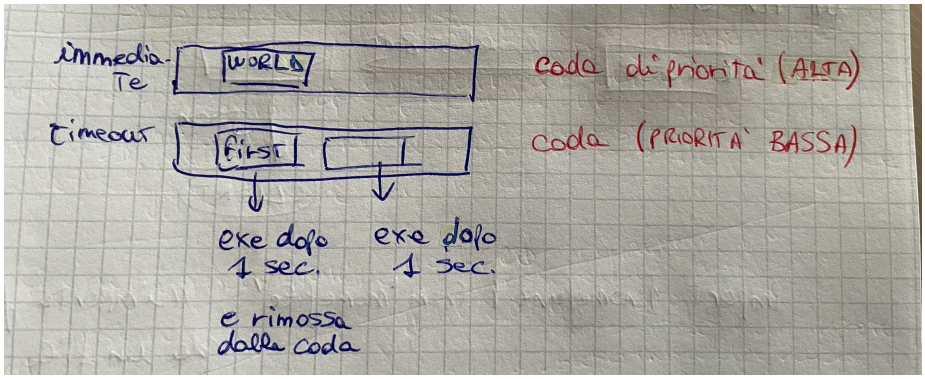
10.2.5 Considerazioni su un esempio di progetto (Node.js simple http server)

La funzione *require()* è fornita da Node.js infatti nel Javascript puro non è presente, la sintassi per utilizzarla correttamente è la seguente:

```
const mymod = require('nomefile');
```

Differenza tra setTimeout() vs setImmediate()

- **setTimeout()** : esegue la funzione (callback) passata come parametro ogni tot tempo in base a quanto specificato in millisecondi. La callback viene prima inserita in una coda di priorità ed eliminata dalla coda quando è il momento della sua esecuzione (Asincrona)
- **setImmediate()** : esegue la funzione(callback) passata come parametro dopo averla inserita anch'essa in una coda di priorità, ma questa è una coda di ALTA PRIORITÀ. Anche questa funzione è NON BLOCCANTE (Asincrona)



N.B: Se ho funzioni lente all'interno dello script (come quelle che fanno un uso intenso della CPU), esse rallentano l'intero script.

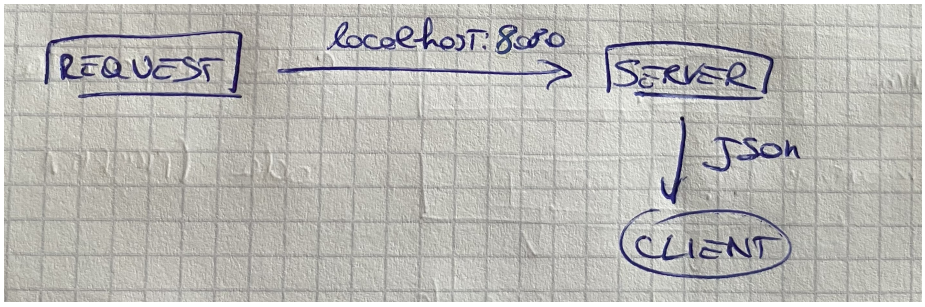
Nel file package.json è possibile trovare alcune impostazioni come:

- **"1.4.0"** : nelle dependences queste stringhe indicano la versione utilizzata di una libreria, se prima del numero si trova il simbolo della potenza allora indica da quella versione e successive.
- **"devDependences"**: indicano le dipendenze riguardo alla compilazione del progetto come la versione del compilatore di Typescript.

Inoltre all'interno della root del progetto posso trovare file come:

tsconfig.json è il file di configurazione del compilatore Typescript.
npx tsc : esegue in Console il compilatore per il programma Typescript e crea i

rispettivi file Javascript.Per questo motivo è bene eseguirlo prima di node perchè altrimenti avrò errori da parte dell'interprete Javascript.
node nomefile: esegue i file Javascript anche quelli creati dal compilatore di Typescript. Per fare i test sul funzionamento del web server senza avere a disposizione un frontend si possono utilizzare software come **Postman** che simula esattamente cosa farebbe un web server, in esso è possibile anche importare file con alcuni test-case.



N.B: Quando c'è una grande quantità di dati da inviare al web server è obbligatorio dividere i dati in pezzi(chunk) più piccoli e inviati al server uno ad uno in maniera asincrona.

Questa è la parte più importante del codice perchè gestisce tutte le richieste che arrivano al web service e fornisce una risposta.

```
req.on("data", (...) => {
        ...                 /*eseguita ogni volta che il
                              client invia i dati*/
    }).on("end", () => {
        ....                /*eseguita ogni volta che non
                              ci sono ulteriori dati da
                              inviare, quindi crea la
                              risposta*/
    });
});
```

Inoltre la seguente riga che consente la cancellazione dei messaggi, utilizza il parametro *"query"* che cerca nel query section dell'url.

```
url.parse("...").query; /*cerca nella query section
                          dell'url passato come parametro
                          quale messaggio eliminare */
```

11 MongoDB

Tutte le richieste a MongoDB usano le **promises** per essere eseguita in maniera asincrona.
È un **DBMS**:

- **Non relazionale**
- **Orientato ai documenti** (struttura simile al file system di un sistema operativo)
- **Con uno schema dinamico**(non definisco uno schema fisso di come salvare i dati)
- **I documenti sono il formato JSON**.

Inoltre **si integra** facilamente **con linguaggi dinamici** come Javascript e la **struttura del database può cambiare al volo** secondo le modifiche volute.

- Ogni **database** è **composto** da **collections** (simile alla tabelle dei database relazionali)
- Ogni **collection** è un **insieme** di **documenti**
- Ogni **documento** è **composto** da **uno** o **più campi**. Come nel seguente esempio:

```
1    {
2        "name" : "...",
3        "class" : "...",
4        "address" : {...}
5    }
```

Ci sono 2 modi per interagire con MongoDB in Javascript:

- **Mongoose**: è un modulo Javascript per interagire direttamente con il database e usa **ODM** (Object-Document Mapping) per mappare gli oggetti in documenti da salvare nel database. Permette inoltre di definire uno schema di come sono i documenti JSON e li mappa automaticamente da/per il database.

  I concetti chiave sono:
  SCHEMAS: Descrive la struttura di un documento della collezione
  MODELS: Fornisce funzioni(costruttori) che istanziano oggetti Javascript a partire da uno schema e li salva automaticamente nel database.

```
import mongoose = require('mongoose');
....
const messageSchema = new mongoose.Schema( {
    ....
})
```

N.B: Posso inoltre inserire anche funzioni all'interno della schema oltre alle property dell'oggetto

Di seguito alcune funzioni utili di mongoose:

```
<model>.find({}); // fa una query al database
<model>.create({obj}); // crea e salva un oggetto
<model>.remove({})
<model>.deleteOne({}) /* entrambe servono ad eliminare
                         oggetti esistenti nel db.*/
```

11.1 Promises vs Callbacks

- **Promises**: Sono **funzioni** che **tornano** un **oggetto Javascript** che è possibile **usare nel futuro** (promise).
- **Callbacks**: Funzioni che vengono eseguite dopo che un evento è terminato.

Le promises hanno 3 stati possibili:

- **Pending**: stato inziale
- **Fullfilled**: l'operazione asincrona è completata
- **Rejected**: l'operazione asincrona è fallita (errore)

Per **ottenere i dati** dal database si usano le **callbacks**, ma a differenza delle callbacks tradizionali, queste vengono **inserite all'interno delle promises.**

- **then() associa una callback quando la promise è fullfilled**. Essa ritorna una nuova promise con il risultato della callback. Ecco il motivo per cui le promises possono essere concatenate.
- **catch() eseguita in caso di errore nel then**

Entrambe usano un approccio molto simile ad un blocco try-catch.
Il costruttore di una promise accetta una funzione che ha come parametri (resolve e reject) che sono a loro volta delle funzioni, in cui la prima è da invocare nel caso di successo, la seconda invece quando c'è un errore.

N.B: Posso anche inserire le keyword AWAIT e ASYNC per rendere le promise più leggibili

- **ASYNC**: viene scritta prima della definizione della funzione che torna una promise e la rende asincrona
- **AWAIT**: può essere usata solo all'interno di una funzione asincrona e attende il risultato di una funzione asincrona. NON BLOCCA L'INTERPRETE, ma il codice scritto dopo viene copiato all'interno di una funzione che funge da handler della promise.

N.B: Nel caso di **funzioni asincrone all'interno** del blocco **try-catch** è importante **utilizzare la keyword await** altrimenti perdo l'eventuale eccezione che può essere lanciata.

12 ExpressJS

È un **modulo di NodeJS** che **fornisce** un **middleware** che serve per **gestire** le **routing** verso gli endpoint.

Il **middleware** (in Express) è una **funzione che prende come parametri una richiesta e una risposta**, le modifica secondo ciò che deve eseguire **e invoca il prossimo middleware**, o **termina con una risposta**. Posso creare quindi una **catena di middleware**.
Inoltre express contiene alcuni middleware built-in come funzioni per il parsing dei file JSON, Data compression, ecc..

12.1 Routing

Specifica delle regole su come mappare gli endpoints ai rispettivi middleware, usando regexp semplificate come:

```
app.<method>(<path>,[<handlers>])
```

- <**method**>: indica il metodo HTTP della richiesta
- <**path**>: indica l'URL dell'endpoint sottoforma di regexp
- <**handler**>: indica la funzione da eseguire quando arriva tale richiesta

N.B: È inoltre raccomandato per gestire le eccezioni chiamare manualmente il metodo next(), passando direttamente un oggetto che descrive l'errore catturato.

12.2 Static pages

Esistono in Express.js delle **funzioni built-in per creare e inviare pagine statiche**.

Ma come salvare le password nel DB?
REGOLA 1: NON SALVARE LE PASSWORD IN CHIARO NEL DB.
Un'**alternativa** potrebbe essere **salvare l'hash della password**, ma anche questa alternativa non è così sicura, perchè è possibile controllare se quell'hash corrisponde all'hash di una delle password più comuni.
Posso però **salvare il salt**, ossia **concatenare la password con una stringa casuale** e poi **calcolare l'hash** della concatenazione.

12.3 Autenticazione con Express.js

In una richiesta di autenticazione come la **basic authentication** vengono fornite nell'header lo username e la password.
Se entrambi sono corretti verrà generato un token per l'accesso che verrà, successivamente, inserito nell'header di tutte le richieste che richiedono l'autenticazione (come l'autenticazione di tipo Bearer token).
In Express per gestire l'autenticazione esistono varie librerie, ma la più usata è *passport-http*, utilizzata in questo modo

```
passport.authenticate();
```

Per verificare il corretto inserimento delle credenziali, **passport guarda nell'header della richiesta**, **decodifica il token** e **separa username e password**. Infine invoca la funzione **done**() che ritorna una promise con i dati dell'utente autenticato, oppure una promise contenente l'eventuale errore. Se l'autenticazione va a buon fine allora invoca il prossimo middleware.
N.B: Passport è in grado anche di gestire le sessioni attraverso i cookie.

Nel token è possibile inserire nel payload alcuni dati rilevanti come nome, ruolo, ecc..., ma NO PASSWORD!!! Per quest'ultima è una buona idea salvarla in una variabile d'ambiente del progetto (cartella env).
Una volta generato il token esso viene firmato dal server. In caso di modifiche esso non sarà più valido perchè la firma non sarà più la stessa.

Nelle **successive richieste** in cui è obbligatoria l'autenticazione, il **client invia il token**, il **server** tramite un middleware ne **controlla la firma** e se essa è **valida** allora **salva il token in req.auth e prosegue**, altrimenti in caso **non sia valida** la firma **oppure** il **token è scaduto darà errore**.
Inoltre il token offre il vantaggio di poter fare controlli senza dover toccare il database.

Un'altra libreria utile è **Socket.io** che **implementa** il **web socket protocol**, passando lo stesso server usato per lo sviluppo dell'app. Serve a **fornire notifiche al client in maniera asincrona** quando un messaggio viene postato (vedi esempio completo). Utile per il frontend.

13 Angular

Angular è un **framework** utilizzato per lo **sviluppo frontend** di web clients SPA(Single Page Application) è **open source** ed è **scritto** in **Typescript**.
In un progetto Angular è possibile trovare:

- Un singolo file HTML(Single Page)
- Più file Javascript

13.1 MVC

Angular utilizza come **design pattern** il **Model-View Controller** (MVC) che consiste nel separare l'app in 3 parti:

- **Model**
- **View**
- **Controller**

l'obiettivo è quello di **separare** il **data model** dalla **logica di presentazione dei dati**. Questo design pattern è molto utile proprio nell'ambito dello sviluppo di GUI (Graphical User Interface).

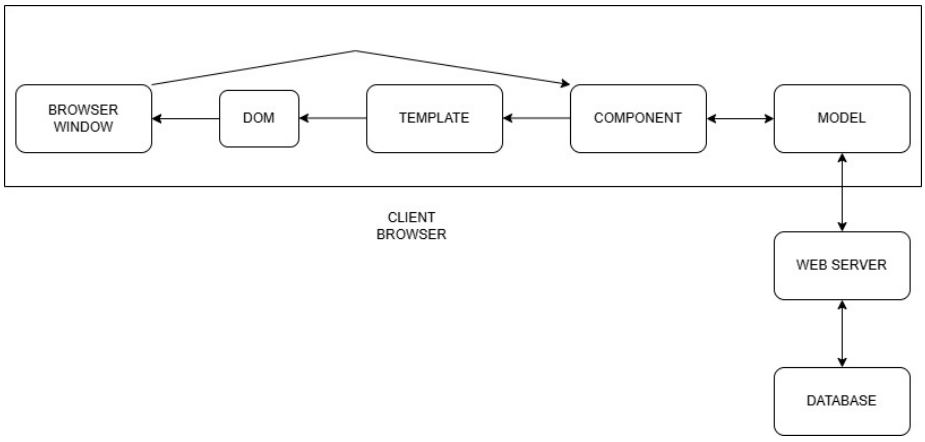


Figure 4: Struttura Angular

13.1.1 Model

- Contiene i dati di un particolare dominio
- **Contiene** la logica per creare, modificare e gestire dati
- **Fornisce API** per esporre i dati e lavorare su di essi

- **Non deve esporre i dettagli** di come ottiene e gestisce i dati
- **Non deve contenere** la logica di come manipola e trasforma i dati in base all'interazione con l'utente
- **Non deve contenere** la logica di come visualizzare i dati

13.1.2 Controller

- **Contiene** la logica che consente di modificare il modello in base all'interazione con l'utente
- **Contiene** la logica di come impostare lo stato iniziale delle view
- **Contiene le feature richieste dal template** per accedere ai dati
- **Non deve gestire** i dati da visualizzare
- **Non deve gestire** come conservare i dati, task che richiama al web service

13.1.3 Template (View)

- **Contiene la logica markup**, necessaria per presentare i dati all'utente
- **Non deve modificare i dati**. Solo visualizzarli
- **Non deve contenere la logica di sviluppo** dell'applicazione

13.2 Decorators

Angular usa molto i decoratori per annotare classi, metodi e proprietà. Essi **servono per aggiungere metadati e feature a runtime.** Un decoratore può dipendere da parametri valutati a runtime.

```
function sealed(constructor: Function) {
    Object.seal(constructor);
    Object.seal(constructor.prototype);
}

@sealed // this is a decorator
class Greeter {
    greeting: string;
    constructor(message: string) {
        this.greeting = message;
    }
    greet() {
        return "Hello, " + this.greeting;
    }
}
```

13.3 Architettura di Angular

NgModule è un **container che contiene componenti**, **direttive** e **servizi** utili per Angular.
Ogni app Angular ha almeno un NgModule che prende il nome di "root module" che definisce come "inizia" l'app ossia cosa mostrare. Un modulo Angular può importare ed esportare funzioni per/da altri moduli. **Un modulo è una classe Angular che è decorata con @NgModule**.
Il decoratore @NgModule prende come input un oggetto che definisce i metadati del modulo:

- Declarations
- Imports
- Exports
- Providers

La differenza tra un modulo Angular e un modulo Typescript è che il **primo** si tratta di una **semplice classe che è annotata con @NgModule** , il **secondo** è un **file .ts che contiene codice Typescript e che usa import come keyword per importare funzioni.**

13.4 Angular Components

Gli **Angular components sono responsabili della visualizzazione dei dati dell'applicazione, attraverso il controllo di una porzione dello schermo (DOM)** e consentirne l'interazione con essi.
Ogni componente ha:

- **Una classe Typescript** che ne gestisce il comportamento e ne contiene i dati
- **Un template** che gestisce come visualizzare i dati definiti dal decoratore @Component. Il template utilizza HTML e CSS

13.5 Data Binding

Angular fornisce 3 modi diversi per il data binding che usano il **meccanismo Markup binding**:

- **Interpolation**: Inserisce la stringa valutata da un'espressione in codice HTML, racchiusa tra parentesi graffe.
- **Property binding**: Mapping di una property del DOM alla classe basandosi sulla valutazione di un'espressione.
- **Event Binding**: Mapping di un evento del DOM nell'invocazione di una funzione specifica nella classe.

N.B: Ogni volta che avviene una **modifica**, Angular **esegue il parsing del template HTML** e dopo aver valutato ogni espressione **genera il nuovo file HTML definitivo.**

Però questi 3 modi hanno anche alcune restrizioni. Le principali sono:

- **L'interpolation deve essere idempotente**, ossia ogni espressione deve essere valutata senza che cambi lo stato dell'applicazione
- Stessa restrizione vale anche nel caso della **property binding**
- **Non è possibile accedere a oggetti al di fuori del contesto definito dal template**. Ad esempio non si possono invocare librerie Javascript all'interno del template HTML.

13.6 Built-in directives

Le directives sono dei costrutti speciali(tag html) di Angular applicabili su elementi del DOM. Di seguito alcuni esempi:

- *ngIf: include un elemento e il suo contesto solo se l'espressione presente nell'if è vera
- *ngFor: consente di ripetere un certo fragment molte volte, iterando su una collezione di elementi del DOM

È inoltre possibile usare delle variabili riferendosi ad elementi del DOM.

13.7 Comandi per installare Angular

Da CLI:

npm install -g@angular/cli

genera la struttura di una nuova app Angular

ng new <app name>

genera una nuova app vuota

ng generate component <comp-name>

aggiunge un componente all'app

ng serve –host 0.0.0.0

esegue l'app all'interno di un container accettando tutti gli indirizzi IP possibili.

13.8 Dependency injection

La **Dependency injection** è un pattern molto importante e molto usato in Angular framework perchè **gestisce le dipendenze tra i vari componenti Angular**. Ecco un esempio di un problema che si avrebbe senza dependency injection

```
export class Car{
    public engine: Engine;
    public tires : Tires;

    constructor() {
        this.engine = new Engine();
        this.tires = new Tires();
    }
}
```

In questo caso la classe Car è "fragile" perchè dipende dalle interfacce che definiscono Engine e Tires e anche dalle loro istanze.

```
export class Car{
    constructor(public engine: Engine, public tires: Tires){}
}
```

Con questa soluzione la classe Car dipende solo dall'interfaccia degli oggetti che usa. Senza istanziarli.
Angular fa questo in automatico tramite un componente chiamato **Injector** che si occupa di:

- gestire una lista di 'injectable objects' come dependences
- sa come costruire ogni oggetto
- può istanziare ogni injectable object quando serve

In Angular, un **service** è una **classe che può essere injected** in un componente per fornire feature specifiche, indipendentemente da come vengono visualizzati i dati.
Ogni service è una classe decorata con il decoratore **@Injectable**. **Nel processo di bootstrap, Angular crea un injector service** che può essere:

- inserito all'interno del singolo componente
- inserito in un @NgModule e condiviso tra tutti i componenti

In entrambi i casi Angular lo mantiene in memoria per poterlo riutilizzare ogni volta che ce n'è bisogno.

13.9 Observables

Un altro importante design pattern sono gli Observable che **permettono di gestire lo scambio di dati in maniera asincrona**.
Un **Observable definisce una funzione che può pubblicare dei valori** ad un oggetto chiamato Observer.
Un **Observer** è un **oggetto che contiene 3 metodi**:

- **next()**: handler che gestisce ogni valore generato
- **error()**: gestisce gli eventuali errori
- **complete()**: viene invocato quando non ci sono ulteriori valori da pubblicare.

N.B: La creazione di un Observable non esegue le funzioni associate. Per eseguire effettivamente le funzioni associate bisogna invocare il metodo **subscribe()**.

13.9.1 Differenza tra Observables e Promises

Di seguito alcune differenze tra Observables e Promises:

- Gli Observable sono dichiarativi e consentono di creare delle "ricette" che possono essere eseguite quando un componente invoca subscribe(). Le promises vengono eseguite, invece, appena vengono create;
- Gli Observable pubblicano una sequenza di valori. Le promises ne pubblicano solo uno, ossia l'oggetto contente la risposta ad una richiesta;
- Gli Observable vengono eseguiti solo dopo l'invocazione del metodo subscribe() e possono creare una catena di trasformazioni sui valori pubblicati(simile al lavoro che può fare un iteratore);
- Si può fermarne l'esecuzione invocando il metodo unsubscribe().

13.10 HttpClient

Una delle classi più usate in Angular è **HttpClient**. Essa è una classe built-in di Angular e **consente di eseguire richieste HTTP asincrone ad un web service e ritornare la risposta sottoforma di Observables**.
Alcuni metodi come get() e delete() si aspettano come parametri dati formattati come JSON string, ed è possibile definire header addizionali, aggiungere parametri alla richiesta ecc..
Ecco come utilizzarla:

- Inject HttpClient service nel costruttore del componente in cui si vuole utilizzare tale service

```
constructor(private http:HttpClient){}
```

- Fai la richiesta

  ```
  http.get(<url>:string , <options>:{}) : Observable<any>
  ```

- Per inviare i dati al server si possono usare metodi come post e put

  ```
  http.post(<url>:string , <body>:any , <options>:{}) : Observable<any>
  ```

13.11 Routing

Angular **consente di dividere l'app in più pagine virtuali differenti** tramite un componente chiamato Router indicando l'URL, come se si navigasse in Internet, **anche se la pagina web non viene mai ricaricata, solo modificata**.

13.11.1 Setup routing

I passaggi da seguire per impostare una routing sono i seguenti:

- **Specificare qual è il base URL** che puo essere definito nel template dal tag:

  ```
  <base href="/">
  ```

  da questo URL vengono poi composti tutti gli URL per le sottoview. Il base URL viene definito in index.html file

- **Viene generato un modulo e aggiunto al progetto** tramite il comando

  ng generate module app-routing –flat –module=app

  In cui: app-routing: indica il nome del modulo
  –flat indica di non generare una cartella dedicata al modulo,
  – module=app importa il nuovo modulo in "app" module (che corrisponde al root module)

- **Le route vengono istanziate dall'oggetto Route object** e ogni componente visualizzato ha una propria route specifica (con route s'intende l'URL della pagina virtuale, NON il backend)

- A questo punto posso inserire il **tag <router-outlet>** che si occupa dello **scambio tra i vari componenti dell'app Angular.** In altre parole quale component visualizzare.

- **Per cambiare la view**(navigazione) è possibile inserire il **tramite codice HTML** con il seguente tag:

  ```
  <a routerLink="/nomeRoute"></a>
  ```

oppure tramite **codice Typescript** nel component

```
constructor(private router:Router) {}
change_page(){
    this.router.navigate(['/pageName']);
}
```